

IEnumerable and IQueryable in C# with Examples

In this article, I will discuss **IEnumerable** and **IQueryable** in C# with Examples. Please read our previous article discussing [How to write LINQ Queries](#) with Examples. In C#, **IEnumerable** and **IQueryable** are two interfaces that are frequently used for data manipulation and querying, but they serve different purposes and have distinct behaviors.

Example to Understand IEnumerable and IQueryable in C#

Let us understand these two interfaces with examples. Please look at the following program we wrote using the LINQ Query Syntax in our previous article.

```
class Program
{
    static void Main(string[] args)
    {
        List<int> integerList = new List<int>()
        {
            1, 2, 3, 4, 5, 6, 7, 8, 9, 10
        };

        var QuerySyntax = from obj in integerList
                           select obj;

        foreach (var item in QuerySyntax)
        {
            Console.Write(item + " ");
        }

        Console.ReadKey();
    }
}
```

In the above example, we use the **var** keyword to create the variable and store the result of the LINQ query. So, let's check what the type of variable is. To check this, just mouse over the pointer onto the QuerySyntax variable, and you will see that the type is **IEnumerable<int>**, which is a generic type. So, it is important to understand what is **IEnumerable<T>**.

So, in the above example, instead of writing the **var** keyword, you can also write **IEnumerable<int>**, and it should work as expected, as shown in the below example.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    List<int> integerList = new List<int>()
    {
        1, 2, 3, 4, 5, 6, 7, 8, 9, 10
    };
    IEnumerable<int> QuerySyntax = from obj in integerList
                                   where obj > 5
                                   select obj;

    foreach (var item in QuerySyntax)
    {
        Console.Write(item + " ");
    }
    Console.ReadKey();
}
}
}

```

With this kept in mind, let us understand what IEnumerable is.

What is IEnumerable in C#?

IEnumerable in C# is an interface that defines one method, GetEnumerator, which returns an IEnumerator object. This interface is found in the **System.Collections** namespace. It is a key part of the .NET Framework and is used to iterate over a collection of objects. The following is the definition of the IEnumerator interface.

```

namespace System.Collections
{
    ... public interface IEnumerator
    {
        ... IEnumerator GetEnumerator();
    }
}

```

GetEnumerator Method:

This is the only method defined in the IEnumerable interface. It returns an IEnumerator object, which provides the ability to iterate through the collection by exposing a Current property and MoveNext() and Reset() methods.

- **Current:** A property that gets the current element in the collection.
- **MoveNext():** This advances the enumerator to the next element of the collection.
- **Reset():** Sets the enumerator to its initial position, which is before the first element in the collection.

IEnumerable is typically used with a foreach loop in C#. The foreach loop automatically uses the GetEnumerator method and the IEnumerator object to iterate over the elements of the collection.

Key Characteristic of IEnumerator in C#:

- **Purpose:** It provides a simple iteration over a collection of a specified type. It's primarily used for in-memory collections like arrays, lists, etc.
- **Execution:** When you use LINQ methods on an IEnumerable, the query is executed in the client's memory. This means all the data is loaded into memory from the data source (like a database), and the operation is performed.
- **Methods:** The extension methods for IEnumerable are defined in the System.Linq.Enumerable class.
- **Deferred Execution:** It supports deferred execution, but the query logic is executed locally on the client side.
- **Use Case:** Best suited for working with in-memory data where the dataset is not too large.

Example to understand IEnumerable with Complex Type using C#:

Whenever we want to work with in-memory objects, we need to use the IEnumerable interface, and the reason for this will be discussed in our [next](#) article. In the below example, we have a complex type called Student. Within the Main method, we created a collection to hold a list of students. Then, we are required to display only the Male students. For this, we are using the LINQ Query to fetch the Male students from the student collection data source, and then using a for each loop, we are iterating through the IEnumerable<Student> collection.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> studentList = new List<Student>()
            {
                new Student(){ID = 1, Name = "James", Gender = "Male"},
                new Student(){ID = 2, Name = "Sara", Gender = "Female"},
                new Student(){ID = 3, Name = "Steve", Gender = "Male"},
                new Student(){ID = 4, Name = "Pam", Gender = "Female"}
            };
            //Linq Query to Fetch all students with Gender Male
            IEnumerable<Student> QuerySyntax = from std in studentList
                                                where std.Gender == "Male"
                                                select std;

            //Iterate through the collection
            foreach (var student in QuerySyntax)
            {
                Console.WriteLine( $"ID : {student.ID}   Name : {student.Name}"
            }
            Console.ReadKey();
        }
    }
    public class Student
```

```

{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
}
}

```

When we execute the program, the result will be displayed as expected, as shown in the image below.

```

ID : 1  Name : James
ID : 3  Name : Steve

```

What is IQueryable in C#?

IQueryable in C# is an interface that is used to query data from a data source. It is part of the System.Linq namespace and is a key component in LINQ (Language Integrated Query). Unlike IEnumerable, which is used for iterating over in-memory collections, IQueryable is designed for querying data sources where the query is not executed until the object is enumerated. This is particularly useful for remote data sources, like databases, enabling efficient querying by allowing the query to be executed on the server side. The following is the definition of the IQueryable interface.

```

namespace System.Linq
{
    ...public interface IQueryable : IEnumerable
    {
        ...Expression Expression { get; }
        ...Type ElementType { get; }
        ...IQueryProvider Provider { get; }
    }
}

```

Here are the key aspects of IQueryable:

- **Deferred Execution:** IQueryable defers the execution of the query until the queryable object is actually iterated over. This means the query is not executed when defined but when the results are required.
- **Expression Trees:** IQueryable queries are represented as expression trees. An expression tree is a data structure that represents code in a tree-like format, where each node is an expression, such as a method call or a binary operation. This allows the query to be translated into a format that can be understood by the data source, such as SQL for a database.
- **Query Providers:** IQueryable relies on an implementation of the IQueryProvider interface to execute queries. The provider translates the expression tree into a format that can be executed against the data source.

Key Characteristic of IQueryable in C#:

- **Purpose:** It is intended to query data from out-of-memory sources, like a database or web service. It is a powerful feature for LINQ, SQL, and Entity Framework.
- **Execution:** The query logic is translated into a format suitable for the data source (like SQL for a relational database). The query is executed on the server side, which can improve performance and reduce network traffic.
- **Methods:** The extension methods for IQueryable are defined in the System.Linq.Queryable class.
- **Deferred Execution:** Supports deferred execution, and the query is executed in the data source (like a database).
- **Use Case:** This is ideal for remote data sources, like databases, where you want to leverage server-side resources and minimize data transfer.

Example to Understand IQueryable<T> Interface in C#.

The following is the same example as the previous one. Instead of IEnumerable<Student>, we store the result in the IQueryable<Student> variable. To store the result in the IQueryable<Student> variable, we need to call the AsQueryable() method on the data source.

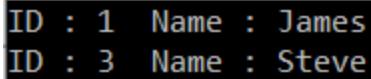
```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> studentList = new List<Student>()
            {
                new Student(){ID = 1, Name = "James", Gender = "Male"},
                new Student(){ID = 2, Name = "Sara", Gender = "Female"},
                new Student(){ID = 3, Name = "Steve", Gender = "Male"},
                new Student(){ID = 4, Name = "Pam", Gender = "Female"}
            };

            //Linq Query to Fetch all students with Gender Male
            IQueryable<Student> MethodSyntax = studentList.AsQueryable()
                .Where(std => std.Gender == "Male");

            //Iterate through the collection
            foreach (var student in MethodSyntax)
            {
                Console.WriteLine( $"ID : {student.ID}   Name : {student.Name}"
            }
            Console.ReadKey();
        }
    }
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
    }
}
```

```
        public string Gender { get; set; }  
    }  
}
```

Now run the application, and you will see the data as expected, as shown in the below image.



```
ID : 1 Name : James  
ID : 3 Name : Steve
```

The point you need to remember is to return a collection of `IQueryable` type and call the **`AsQueryable()`** method on the data source, as we did in the above example.

Key Differences Between `IEnumerable` and `IQueryable` in C#

- **Execution Context:** `IEnumerable` executes in the client memory, whereas `IQueryable` executes on the data source.
- **Suitability:** `IEnumerable` is suitable for LINQ to Objects and working with in-memory data. `IQueryable` is suitable for LINQ to SQL or Entity Framework to interact with databases.
- **Performance:** `IQueryable` can perform better for large data sets as it allows the database to optimize and filter data.

Choosing Between `IEnumerable` and `IQueryable` in C#:

- Use `IEnumerable` when working with in-memory data collections where the data set is not excessively large.
- Use `IQueryable` when querying data from out-of-memory sources like databases, especially when dealing with large data sets, to take advantage of server-side processing and optimizations.

In the next article, I will discuss the [Differences Between `IEnumerable` and `IQueryable` in C#](#) and when to use one over another with an example. I hope this article gives you a good understanding of two important interfaces, i.e., `IEnumerable` and `IQueryable` in C# with Examples.